VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belgaum – 590014

A
Mini Project Report on

"Automated Attendance System using ESPov5640 CAM "

A Dissertation work submitted in partial fulfilment of the requirement for the award of the degree

Bachelor of Engineering
In
Electrical and Electronics Engineering

Submitted by

VAISHNAVI M	**1AY22EE046**
YASHWANTH RAJU	**1AY22EE050**
BHAVANISHANKAR	**1AY22EE403**

Under the guidance of
Mr.Karthik S
Asst. Professor

Department of

Electrical and Electronics Engineering

Acharya Institute of Technology

(An Institute Affiliated to Visvesvaraya Technological University, Belgaum, Recognized by AICTE,)

Acharya Dr. Sarvepalli Radhakrishnan Road, Soldevanahalli, Bangalore-560107

2024 - 2025

ACHARYAINSTITUTE OF TECHNOLOGY

SOLDEVANAHALLI, BENGALURU – 560107

DEAPARTMENT OF ELECTRICAL AND ELECTRONICS ENGINEERING

2024-2025

CERTIFICATE

This is to certify that the Mini project work entitled "AUTOMATED ATTENDANCE SYSTEM USING ESP ov5640 CAM" is carried out by **YASHWANTH RAJU (1AY22EE050)**, **BHAVANI SHANKAR (1AY22EE403)**, **VAISHNAVI M (1AY22EE046)** bonafide students of **ACHARYA INSTITUTE OF TECHNOLOGY** in partial fulfilment for the internal assessment of 5th Semester of the ability Enhancement course subject "MINI PROJECT – BEE586" for **Bachelor of Engineering in Electrical and Electronics Engineering** of the **Visvesvaraya Technological University, Belagavi** during the academic year **2024-2025**

_____	_____	_____
Signature of Guide	**Signature of H.O.D**	**Signature of Principal**
Mr.Karthik S	**Dr. Shashidhar T.M.**	**Dr. Rajeswari**
Asst.Prof	**Prof. and HOD**	**Principal**
Dept of EEE, AIT, Bengaluru	**Dept of EEE, AIT, Bengaluru**	**AIT, Bengaluru**

Name of Examiner	Signature with Date
1. _____	_____
2. _____	_____

TABLE OF CONTENT

INDEX	PAGE NO
ABSTRACT	1
CHAPTER – 1	
1.1 INTRODUCTION	2
CHAPTER – 2	
2.1 LITERATURE SURVEY	3-4
CHAPTER – 3	
3.1 CIRCUIT DIAGRAM	5
CHAPTER – 4	
4.1 HARDWARE IMPLEMENTATION	6-7
CHAPTER – 5	
5.1 SOFTWARE IMPLEMENTATION	8-10
CHAPTER – 6	
6.1 PROPOSED METHODOLOGY	11-12
CHAPTER – 7	
7.1 RESULT & CONCLUSION	13-14
CHAPTER – 8	
8.1 ADVANTAGES	15
CHAPTER – 9	
9.1 FUTURE SCOPE OF SCOPE	16-17
9.2 REFERENCES	18
CHAPTER – 10	
10.1 APPENDIX	19-27

DECLARATION

We VAISHNAVI M(1AY22EE046), YASHWANT RAJU(1AY22EE050), and BHAVANISHANKAR(1AY23EE403), students of V Semester B.E in the Department of Electrical and Electronics Engineering, AIT, Bengaluru, hereby declare that the project entitled" **Automated Attendance System using ESPov5640 CAM** " has been carried out under the supervision of Prof. Karthik S, Asst.Professor, Dept. of EEE, AIT. This project is submitted in partial fulfilment of the requirements for the award of the degree of Bachelor of Engineering in Electrical and Electronics Engineering of Visvesvaraya Technological University, Belagavi, during the academic year 2024-2025. This project has been conducted with dedication, adherence to academic guidelines, and alignment with the ethical standards set by our institution. We extend our gratitude to Prof. Karthik S for his valuable guidance, encouragement, and insightful feedback throughout this project. Additionally, we express our appreciation to the Department of Electrical and Electronics Engineering at Acharya Institute of Technology for providing the necessary resources and support to make this project successful. We acknowledge the collaborative effort and teamwork that has driven this project forward, enabling us to explore innovative solutions and expand our knowledge in the field of robotics and wireless technologies

Ms. VAISHNAVI M (1AY22EE046)

Mr. YASHWANTH RAJU (1AY22EE050)

Mr. BHAVANI SHANKAR (1AY22EE403)

PLACE : BENGALURU

DATE :

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompany the successful completion of a task would be incomplete without the mention of the people who made it possible and without their constant guidance and encouragement success would not have been possible.

I am grateful to the institute **Acharya Institute of Technology management** for its ideas and inspiration for having provided us with the good infrastructure, laboratory, facilities, and inspiring staff which has made this Mini-Project successfully.

We would like to express my greatest appreciation to my guide **Mr. Karthik S** Asst. Professor, Department of Electrical and Electronics Engineering for her encouragement and guidance, timely suggestion during the completion of the project.

We would like to express my sincere thanks to **Dr. Shashidhar T M ,** HOD, Dept. of Electrical and Electronics Engineering, AIT, for his valuable guidance and support.

I would like to express my sincere gratitude to **Dr. Rajeswari** Principal, AIT and Prof**. Mari Gowda C K**, Vice Principal, AIT for all the facilities that has been extended by them throughout my work.

I would like to express my sincere gratitude to the Mini-Project Coordinators **Mr. Lakshmikanth Reddy,** Asst.Professor Dept. of EEE, AIT for their valuable guidance and support.

Finally, I would like to express my sincere gratitude to my parents, all teaching and non- teaching faculty members, and friends for their moral support, encouragement, and help throughout the completion of the Mini-Project.

Abstract

This project aims to develop an automated attendance system utilizing face recognition technology integrated with an ESP32 camera. The system captures real-time images, detects and identifies faces using pre-stored images, and marks attendance automatically by recording the individual's name and timestamp in a CSV file. It also captures and saves screenshots of the detected faces for verification purposes. This solution provides an efficient, accurate, and user-friendly method for attendance tracking in environments such as classrooms and offices, reducing the need for manual recording and minimizing errors.

CHAPTER – 1
INTRODUCTION

CHAPTER – 1 INTRODUCTION

In educational institutions and corporate offices, attendance tracking is a critical yet often time- consuming task. Traditional methods of manual attendance recording are prone to errors, time- consuming, and inefficient. To address these challenges, this project proposes an automated attendance system that leverages face recognition technology and the ESP32 camera module to streamline and enhance the process. The system captures real-time images of individuals using the ESP32 camera. It then employs face recognition algorithms to detect and identify faces by comparing them against a pre-stored database of images. Once a face is recognized, the system marks attendance by recording the individual's name and the current timestamp in a CSVfile. Additionally, to ensure verification and auditability, the system captures and saves screenshots of the detected faces.

This project not only automates the attendance process, making it faster and more accurate, but also provides a reliable method to verify attendance records through captured images. By integrating modern face recognition technology with IoT hardware, this project offers a comprehensive solution for attendance management that can be implemented in various settings such as schools, universities, and corporate offices.

CHAPTER – 2
LITERATURE SURVEY

CHAPTER – 2 LITERATURE SURVEY

[1] Automated Attendance System Using Facial Recognition"

This study explored the use of facial recognition technology to automate attendance systems. The system utilized deep learning models, such as convolutional neural networks (CNNs), to identify and verify faces in real time. High-resolution cameras, similar to the OV5640, were employed for accurate image capture. The results demonstrated improved efficiency and reduced manual effort compared to traditional methods.

[2] IoT-Based Smart Attendance System"

The research proposed an Internet of Things (IoT)-based attendance system integrating cameras and cloud servers. The OV5640 camera module was highlighted for its compact design and ability to capture high-resolution images. The system achieved real-time data synchronization and allowed remote access for attendance tracking, showcasing its scalability and usability in educational institutions.

[3]):"Real-Time Face Detection Using Embedded Camera Modules"

This work focused on deploying face detection algorithms on embedded systems using camera modules like the OV5640. The study addressed challenges such as low-light conditions and motion blur. A combination of hardware optimization and software preprocessing ensured efficient performance, making it suitable for attendance systems.

[4] Integration of Machine Learning in Smart Attendance Systems" This paper discussed the integration of machine learning algorithms with hardware components like the OV5640 camera for attendance automation. Techniques such as image preprocessing, feature extraction, and face recognition were utilized. The system demonstrated high accuracy and robustness across diverse environments.

[5] "Designing a Cost-Effective Attendance System with OV5640 Cameras" The study focused on developing a low-cost, high- efficiency attendance system using the OV5640 camera module. Emphasis was placed on hardware-software co-design to optimize resource usage. The system was validated in real-world scenarios, proving its effectiveness for small- to medium-scale deployments.

[6] "Existing Research and Developments"

Facial Recognition-based Attendance Systems: Several studies have explored the use of facial recognition for automated attendance systems. A notable development is the use of Convolutional Neural Networks (CNNs) and deep learning algorithms to process images and match them with pre-stored student or employee facial data. For instance, a study by Gualtieri et al. (2021) proposed an AI-based facial recognition system to automate attendance in universities.

[7] "Comparative Analysis of Facial Recognition Models"

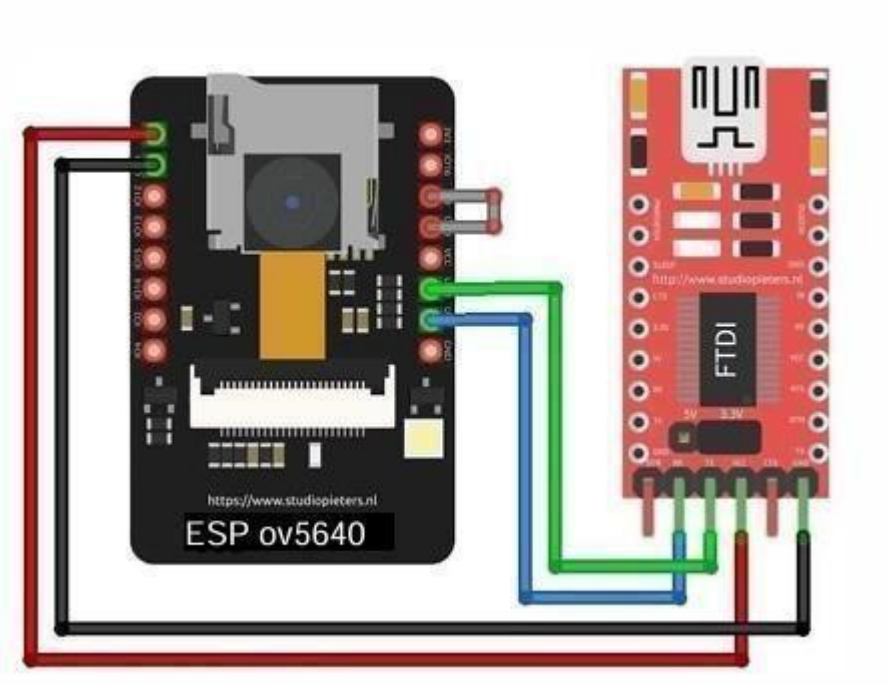
Haar Cascade Classifier vs. Deep Learning Models: Traditional methods like Haar Cascade Classifiers, based on machine learning and feature extraction, have been widely used for face detection in embedded systems. While these methods are relatively lightweight, they tend to perform poorly in complex or uncontrolled environments (e.g., varying lighting, partial occlusion). On the other hand, deep learning models like CNNs (Convolutional Neural Networks) or more recent architectures like MobileNet and FaceNet offer much higher accuracy, though they require more processing power and may need to be run on external servers for real-time performance.

[8] "Recent Developments in Attendance Systems"

Multi-Factor Authentication Systems: Many modern attendance systems combine multiple biometric features such as facial recognition, fingerprint scanning, and RFID for enhanced accuracy and security. Research by Rani et al. (2022) examined an automated attendance system that utilized both facial recognition and RFID for a hybrid authentication approach. The system aimed to minimize false positives in attendance marking by validating both facial and RFID data, providing a more secure and reliable method of attendance tracking.

CHAPTER – 3
CIRCUIT DIAGRAM

CHAPTER – 3 CIRCUIT DIAGRAM



ESP 32 - CAM	FTDI Programmer
GND	GND
5 V	VCC
UOR	TX
UOT	RX
GPIO0	GND

CHAPTER – 4
HARDWARE IMPLEMENTATION

CHAPTER – 4 HARDWARE IMPLEMENTATION

COMPONENTS:

1) ESPov5640 CAM MODULE

The ESP32-CAM module is a cost-effective and versatile solution ideal for camera-based applications. It features an integrated OV2640 camera, built-in Wi-Fi and Bluetooth for wireless communication, and a powerful ESP32 SoC capable of handling real-time image processing tasks like face recognition. Its compact size and expandable storage via a microSD card slot make it suitable for embedded systems. Additionally, a strong community and extensive online resources support developers in creating various projects, from surveillance and home automation to robotics and IoT applications. This combination of affordability, performance, and flexibility makes the ESP32-CAM a valuable component for innovative projects.

The **ESP32-OV5640 Camera Module** is an ideal solution for applications requiring image capture and streaming, especially in IoT systems where **Wi-Fi** or **Bluetooth** connectivity is necessary. Its high-resolution camera, combined with the powerful ESP32 microcontroller, enables a wide range of innovative and efficient applications such as **facial recognition**, **remote monitoring**, and **smart home automation**.

2) FTDI Module

The FTDI module, developed by Future Technology Devices International, is a popular USB-to-serial interface converter. It enables seamless communication between a computer's USB port and serial devices, such as microcontrollers and development boards, by converting USB signals to UART (Universal Asynchronous Receiver-Transmitter) signals. Widely used in electronics prototyping and development, the FTDI module simplifies programming and debugging by providing a reliable and straightforward interface. It supports multiple operating systems and offers features like configurable baud rates and signal integrity, making it an essential tool for embedded systems and hardware interfacing projects.

The FTDI module connects to the **USB** port of a computer and translates the **USB data** into a serial communication format.

The microcontroller or embedded system connects to the FTDI module's **TX** (transmit) and **RX** (receive) pins to facilitate communication

CHAPTER - 5

SOFTWARE IMPLEMENTATION

CHAPTER – 5 SOFTWARES USED

Arduino IDE

The Arduino Integrated Development Environment (IDE) is a versatile and user-friendly platform designed for writing, compiling, and uploading code to Arduino boards. It is the primary tool used by developers and hobbyists to create applications and projects using the Arduino microcontroller platform. The Arduino IDE simplifies the process of programming microcontrollers by providing an intuitive interface and a range of features that cater to both beginners and experienced users.

Here are some key aspects of the Arduino IDE:

Cross-Platform Compatibility: The Arduino IDE is available for multiple operating systems, including Windows, macOS, and Linux, ensuring broad accessibility.

Simple and Intuitive Interface: The interface is designed to be straightforward, with basic features such as a text editor for writing code, a message area, a console for displaying output and error messages, and a toolbar with buttons for common functions like verifyingand uploading code.

Code Compilation and Uploading: The IDE includes a compiler that converts the written code into machine language that the Arduino board can understand. Once compiled, the code can be uploaded to the board via a USB connection written code to expand the functionality of Arduino projects.

PYTHON IDLE

Python IDLE (Integrated Development and Learning Environment) is a user-friendly interface bundled with the standard Python distribution, designed for writing, running, and debugging Python code. It features a code editor with syntax highlighting, auto-completion, and an integrated interactive shell for real-time code testing. IDLE's built-in debugger allows step-by- step execution and variable inspection, making it ideal for both beginners and experienced developers. It simplifies the development process by providing an accessible environment to write and test Python scripts, facilitating learning and rapid prototyping.

• **Code Development:** Python IDLE offers a simple and intuitive environment for writing and editing Python scripts. You can use it to develop the various components of your attendance system, such as image processing, face recognition, and data handling.

Interactive Shell: The interactive shell in IDLE allows you to test individual lines of code or small code snippets immediately. This is particularly useful for experimenting with image processing functions, face recognition algorithms, and debugging.

Debugging: IDLE includes a built-in debugger that helps you step through your code, inspect variables, and identify issues. This is essential for ensuring that your face recognition and attendance marking functions work correctly.

Learning and Prototyping: IDLE is designed to be beginner-friendly, making it a great tool for learning and quickly prototyping new features or modifications in your project.

Visualization: For parts of the project involving data visualization(e.g., displaying attendance logs), IDLE can be used to quickly plot and visualize data, ensuring that the outputs are as expected

CHAPTER – 6

METHODOLOGY

CHAPTER-6 METHODOLOGY

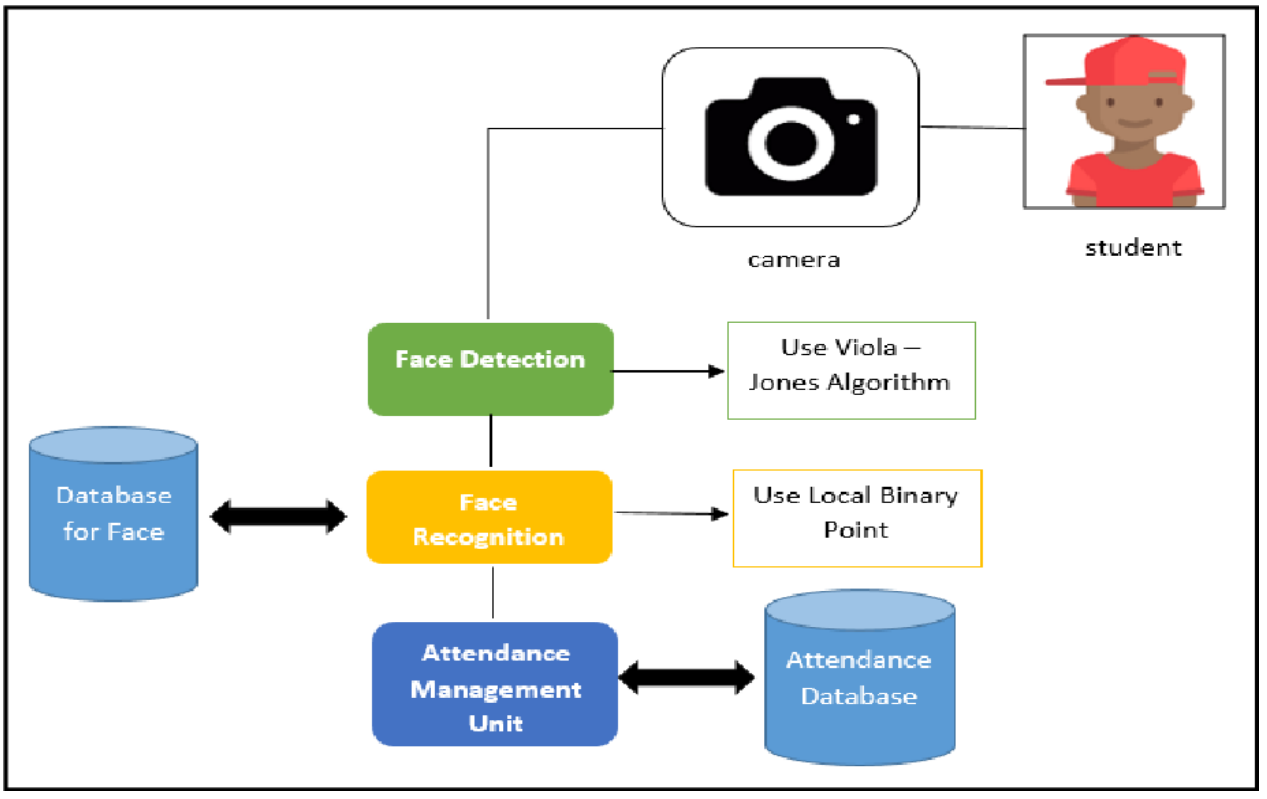


Image acquisition:

The initial step is the image will be captured from a camera that is positioned in the classroom. ideally, the cameras she must be a mia qualit a guaias it in era. the a hiera qualit positra ca a produced of the class where it can capture the image of the whole class. The image should contain all the faces of students that are present in a particular class.

Face detection

In this research, face detection algorithm that will be used is Viola Jones Algorithm. In Viola Jones algorithm, there are four phases that are involved which is Haar feature selection, Integral image, Adaboost

training and cascading [23]. In Haar Feature selection, the similarities of human face are represented by Haar feature. The features are represented by two or three rectangles that are composed together. For example, the human faces have similarities where normally, the nose bridge part will be brighter than the eyes. In this case, Haar features that represent the dark and white area are choosen. By using this technique, it is easier to detect

whether face is present or not in an image.

Face recognition:

Local Binary Pattern (LBP) will be used in order to perform face recognition. By thresholding the surrounding of each pixel with the value of the center pixel, LBP labels the image pixels. The result is considered as binary number [24]. When the bit pattern is circular and the binary pattern contains maximum of two bitwise transitions from O to 1 or the other way around, it is called uniform [25]. In LBP histogram computation, it uses uniform patterns so that the uniform and non-uniform pattern can be assigned to different bin.

Comparison of features:

After the image has been through the face recognition stage, the features will be compared with the stored features. If there are similarities detected, it will be saved in the face database as well as the attendance

Attendance database:

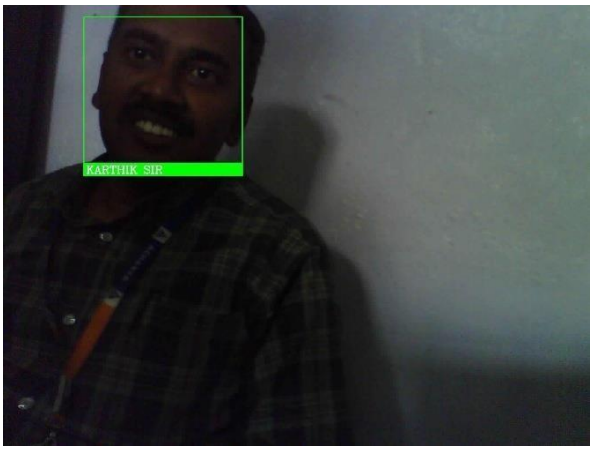
After the comparison has been completed, the attendance will be saved into the attendance database. If a student is present on a particular day, their student id will be saved together with the date and the course
code of the class that the student participates in.

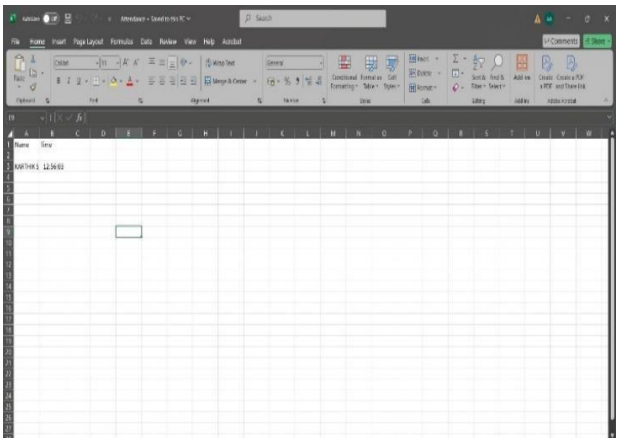
CHAPTER - 7

RESULT AND CONCLUSION

CHAPTER - 7 RESULT

The **Automated Attendance System using ESPov5640 CAM** offers a significant advancement in simplifying attendance tracking in various environments, such as schools, colleges, and workplaces. By leveraging the ESP32 microcontroller and the OV5640 camera module, the system enables efficient and accurate face recognition for automatic attendance marking. This system captures the face of an individual when they stand in front of the camera, processes the image, and compares it against a database of registered faces. If a match is found, the system automatically records the individual's attendance.





The system works by capturing real-time images of individuals as they approach the camera. These images are then processed using facial recognition algorithms to identify the person. Once a match is found, the system logs the person's attendance in a digital format. This data is then stored in an Excel sheet.

The process begins with capturing and storing facial data from authorized individuals in the system's database. These facial features are then used for comparison during each recognition attempt. When a person enters the scene, the camera captures their face and compares it with stored data, determining whether the person is recognized. If recognized, the system marks their presence and records the time and date of their entry. The attendance data is automatically updated in an Excel sheet, noting each person's status as present along with the corresponding timestamp

CONCLUSION

The development of an **Automated Attendance System** using the **ESP32** and **OV5640 Camera Module** has shown to be a practical and efficient solution for real-time attendance tracking. By combining low-cost hardware, facial recognition algorithms, and cloud-based storage, this system offers a modern and scalable alternative to traditional attendance management systems. Below are some key conclusions drawn from building this project:

The **Automated Attendance System** using **ESP32** and **OV5640 Camera** provides an efficient, scalable, and secure solution for modern attendance tracking. The system's ability to integrate facial recognition with cloud databases makes it highly effective for environments like schools, offices, and public institutions. However, careful attention must be given to lighting conditions, camera angles, and facial recognition accuracy to ensure reliable performance. The scalability and flexibility of this system make it a promising solution for various use cases, with potential for further enhancements as technology progresses.

CHAPTER – 8

ADVANTAGES

CHAPTER–8 ADVANTAGES

The attendance system project using face recognition and an ESP32-CAM module offers several advantages including:

Automation: Eliminates the need for manual attendance taking, reducing human error and saving time for both students and administrators.

Accuracy and Reliability: Uses face recognition to accurately identify individuals, ensuring reliable attendance records without the risk of buddy punching.

Real-time Processing: Captures and processes attendance in real-time, providing instant feedback and up-to-date records**.**

Scalability: Easily scales to accommodate more individuals by adding more known face encodings, Making it suitable for small classes or large organizations.

Cost-effectiveness: Utilizes affordable hardware components like the ESP32-CAM, making the solution accessible for educational institutions and small businesses.

Convenience: Reduces the need for physical attendance sheets and manual entry, streamlining the Attendance management process.

Data Management: Automatically stores attendance records in a digital format, facilitating easy access, analysis, analysis, and reporting of attendance data.

CHAPTER – 9
FUTURE SCOPE AND REFERANCE

CHAPTER - 9 Future Scope

The future scope of this attendance system project using face recognition and an ESP32-CAM module includes several potential enhancements and expansions.Here are some key areas where these systems are expected to evolve and have a significant impact:

Integration with Existing Systems: Integrate with existing school or company management systems for seamless data transfer and centralized attendance tracking.

Mobile and Web Applications: Develop mobile and web applications to provide remote access to attendance records, notifications, and real- time monitoring.

Enhanced Security: Implement advanced security features such as encrypted data transmission, secure storage, and multi-factor authentication to protect sensitive information.

Multi-Camera Support: Expand the system to support multiple cameras across different locations, enabling comprehensive coverage and monitoring of larger areas.

Offline Functionality: Add offline capabilities to allow the system to function without an internet connection, synchronizing data when connectivity is restored.

Extended Features: Include additional features like temperature scanning, mask detection, or other biometric verification methods to enhance safety and compliance in specific environments.

Scalability for Large Organizations: Optimize the system for high scalability, allowing it to efficiently manage attendance for large organizations with thousands of employees or students.

Customizable Reports: Enable customizable reporting options to generate detailed attendance reports based on various criteria, such as date range, department, or individual performance.

Integration with IoT Devices: Integrate with other IoT devices, such as smart locks or automated entry systems, to control access based on attendance and identity verification.

CHAPTER – 9 References

[1] R. Kumar and R. Singh, "Face Recognition Techniques: A Comprehensive Study," 2020 6th International Conference on Advanced Computing and Communication Systems (ICACCS), Coimbatore, India, 2020, pp. 945-950, doi: 10.1109/ICACCS48705.2020.9074528.

[2] J. Prabha and V. Priya, "A Comparative Study of Face Recognition Techniques using MATLAB," 2017 International Conference on Computer Communication and Informatics (ICCCI), Coimbatore, India, 2017, pp. 1-5, doi: 10.1109/ICCCI.2017.8117574.

[3] S. S. S. Naidu and N. T. Jadhav, "Face Recognition using PCA and SVM," 2018 International Conference on Computer, Communication, and Signal Processing (ICCCSP), Chennai, India, 2018, pp. 1-5, doi: 10.1109/ICCCSP.2018.8524312.

[4] M. A. Islam and M. R. Islam, "Facial Expression Recognition using Deep Learning Techniques," 2019 22nd International Conference on Computer and Information Technology (ICCIT), Dhaka, Bangladesh, 2019, pp. 1-6, doi: 10.1109/ICCITechn.2019.8966435.

[5] A. Bhatia, M. Singh and N. Singh, "A Comparative Study of Face Recognition Techniques and Databases," 2020 4th International Conference on Inventive Systems and Control (ICISC), Coimbatore, India, 2020, pp. 1-5, doi: 10.1109/ICISC51099.2020.9180875.

[6] A. L. N. Reddy and V. Krishna, "Facial Expression Recognition System Based on Deep Learning," 2020 Fourth International Conference on Computing Methodologies and Communication (ICCMC), Erode, India, 2020, pp. 874-879, doi: 10.1109/ICCMC48892.2020.9219626.

[7] K. Sandeep and M. K. Gandhi, "Facial Emotion Recognition using Machine Learning Techniques: A Survey," 2019 Second International Conference on Intelligent Computing and Control Systems (ICICCS), Madurai, India, 2019, pp. 731-736, doi: 10.1109/ICCONS.2019.8884323.)

CHAPTER – 10

APPENDIX

CHAPTER– 10 APPENDIX:

1. To Program ESP 32 CAM

```
#include <WebServer.h>
#include <WiFi.h>
#include <esp32cam.h>
const char* WIFI_SSID = "Realme5i";
const char* WIFI_PASS = "123456789.";
WebServer server (80);
static auto loRes = esp32cam::Resolution::find (320, 240);
static auto hiRes = esp32cam::Resolution::find (800, 600);
void handleBmp()
{
if (!esp32cam::Camera.changeResolution (loRes)) {
Serial.println("SET-LO-RES FAIL");
}
auto frame = esp32cam::capture();
if (frame == nullptr) {
Serial.println("CAPTURE FAIL");
server.send(503, "", "");
return;
}
Serial.printf("CAPTURE OK %dx%d %db\n", frame
>getWidth(), frame->getHeight(),
static_cast<int>(frame->size()));
if (!frame->toBmp()) {
Serial.println("CONVERT FAIL");
server.send(503, "");
return;
}
Serial.printf ("CONVERT OK %dx%d %db\n", frame
```

```
>getWidth(), frame->getHeight(),
static_cast<int>(frame->size()));
server.setContentLength(frame->size()); server.send(200,
"image/bmp");
WiFiClient client = server.client();
frame->writeTo(client);
}
if (res<=0) {
Serial.printf("STREAM ERROR %d\n", res);
return;
}
auto duration = millis() - startTime;
Serial.printf("STREAM END %dfrm %0.2ffps\n", res, 1000.0 * res /
duration);
}
void setup()
{
Serial.begin(115200);
Serial.println();
{
using namespace esp32cam;
Config cfg;
cfg.setPins (pins:: AiThinker);
cfg.setResolution(hiRes);
cfg.setBufferCount(2);
cfg.setJpeg (80);
bool ok = Camera.begin(cfg);
Serial.println(ok ? "CAMERA OK" : "CAMERA FAIL");
}
WiFi.persistent (false);
WiFi.mode(WIFI_STA);
WiFi.begin(WIFI_SSID, WIFI_PASS);
while (WiFi.status() != WL_CONNECTED) {
```

```
delay(500)
Serial.print("http://");
Serial.println(WiFi.localIP());
//Serial.println("/cam.bmp");
[11:35 am, 09/01/2025] .: Serial.println("/cam-lo.jpg");
Serial.println("/cam-hi.jpg");
//Serial.println("/cam.mjpeg");
//server.on("/cam.bmp", handleBmp);
server.on("/cam-lo.jpg", handleJpgLo);
server.on("/cam-hi.jpg", handleJpgHi);
//server.on("/cam.jpg",  handleJpg):
//server.on("/cam.mjpeg", handleMjpeg);
server.begin();
}
void loop()
{
server.handleClient();
}
void serveJpg()
{
auto frame = esp32cam::capture();
if (frame == nullptr) {
Serial.println("CAPTURE FAIL");
server.send(503, "", "");
return;
}
Serial.printf("CAPTURE OK %dx%d %db\n", frame->getWidth(), frame
>getHeight(),
static_cast<int>(frame->size()));
server.setContentLength (frame->size());
server.send(200, "image/jpeg");
WiFiClientclientserver.client();
```

```
frame->writeTo(client);
}
void handleJpgLo ()
{
if (!esp32cam:: Camera.changeResolution (loRes)) {
Serial.println("SET-LO-RES FAIL");
}
serveJpg();
}
void handleJpgHi ()
{
if (!esp32cam::Camera.changeResolution(hiRes)){
Serial.println("SET-HI-RES FAIL");
}
serveJpg();
}
void handleJpg()
{
server.sendHeader("Location", "/cam-hi.jpg");
server.send(302, "", "");
}
void handleMjpeg()
{
if (!esp32cam:: Camera.changeResolution (hiRes)) {
Serial.println("SET-HI-RES FAIL");
}
Serial.println("STREAM BEGIN");
WiFiClient client = server.client();
auto startTime = millis();
int res = esp32cam:: Camera.streamMjpeg(client);
if (res<=0) {
Serial.printf("STREAM ERROR %d\n", res);
return;}
```

```
auto duration = millis() - startTime;
Serial.printf("STREAM END %dfrm %0.2ffps\n", res, 1000.0 * res /duration);
}
void setup()
{
Serial.begin(115200);
Serial.println();
[11:35 am, 09/01/2025] .: {
using namespace esp32cam;
Config cfg;
cfg.setPins (pins:: AiThinker);
cfg.setResolution(hiRes);
cfg.setBufferCount(2);
cfg.setJpeg (80);
bool ok = Camera.begin(cfg);
Serial.println(ok ? "CAMERA OK" : "CAMERA FAIL");
}
WiFi.persistent (false);
WiFi.mode(WIFI_STA);
WiFi.begin(WIFI_SSID, WIFI_PASS);
while (WiFi.status() != WL_CONNECTED) {
delay(500);
}
Serial.print("http://");
Serial.println(WiFi.localIP());
//Serial.println("/cam.bmp");
Serial.println("/cam-lo.jpg");
Serial.println("/cam-hi.jpg");
//Serial.println("/cam.mjpeg");
//server.on("/cam.bmp", handleBmp);
server.on("/cam-lo.jpg", handleJpgLo);
server.on("/cam-hi.jpg", handleJpgHi);
```

```
//server.on("/cam.jpg",  handleJpg):
//server.on("/cam.mjpeg", handleMjpeg);
server.begin();
}
void loop()
{
server.handleClient();
}
```

2) To run code on PYTHON IDLE

```
import pandas as pd import cv2
import urllib.request
import numpy as np
import os
from datetime import datetime
import face_recognition
# Define paths
base_dir = 'D:\\project espcam\\ATTENDANCE'
attendance_file = os.path.join(base_dir, 'Attendance.csv')
image_folder = os.path.join(base_dir, 'image_folder')
# Ensure the ATTENDANCE directory exists
os.makedirs(base_dir, exist_ok=True)
# Initialize or clear the attendance CSV file
if os.path.exists(attendance_file):
print("Attendance file exists. Removing it.")
os.remove(attendance_file)
# Create a new empty DataFrame and save it to Attendance.csv
df = pd.DataFrame(columns=['Name', 'Time'])
df.to_csv(attendance_file, index=False)
# Load images and class names
images = []
classNames = []
try:
myList = os.listdir(image_folder)
print(f"Images found: {myList}")
for cl in myList:
curImg = cv2.imread(os.path.join(image_folder, cl))
images.append(curImg)
classNames.append(os.path.splitext(cl)[0])
print(f"Class names: {classNames}")
```

```
except FileNotFoundError as e:
[11:42 am, 09/01/2025] .: print(f"Error: {e}")
print("Ensure that the image folder path is correct.")
exit()
def findEncodings(images):
encodeList = []
for img in images:
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
try:
encode = face_recognition.face_encodings(img)[0]
encodeList.append(encode)
except IndexError:
print("No face found in the image.")
return encodeList
def markAttendance(name):
with open(attendance_file, 'r+') as f:
myDataList = f.readlines()
nameList = [line.split(',')[0] for line in myDataList]
if name not in nameList:
now = datetime.now()
dtString = now.strftime('%H:%M:%S')
f.writelines(f'\n{name},{dtString}')
encodeListKnown = findEncodings(images)
print('Encoding Complete')
# URL to fetch the image from the ESP32 camera
url = '
http://192.168.43.55/cam-hi.jpg'
while True:
try:
img_resp = urllib.request.urlopen(url)
imgnp = np.array(bytearray(img_resp.read()),
dtype=np.uint8)
img = cv2.imdecode(imgnp, -1)
except Exception as e:
```

```
print(f"Error fetching image from URL: {e}"
break
    imgS = cv2.resize(img, (0, 0), None, 0.25, 0.25)
    imgS = cv2.cvtColor(imgS,
   cv2.COLOR_BGR2RGB)
   facesCurFrame =
   face_recognition.face_locations(imgS)
   encodesCurFrame =
   face_recognition.face_encodings(imgS,
   facesCurFrame) for encodeFace, faceLoc
   in zip(encodesCurFrame,
   facesCurFrame):
   matches=face_recognition.compare_faces(
   encodeListKnow n, encodeFace)
   faceDis =face_recognition.face_distance(encodeListKnown,
   encodeFace) matchIndex = np.argmin(faceDis)
    if matches[matchIndex]:
    name = classNames[matchIndex].upper() y1, x2, y2, x1 = faceLoc
    y1, x2, y2, x1 = y1 * 4, x2 * 4, y2 * 4,
   x1 * 4 cv2.rectangle(img, (x1, y1),
   (x2, y2), (0, 255, 0), 2)
    cv2.rectangle(img, (x1, y2 - 35), (x2, y2), (0, 255, 0),
   cv2.FILLED) cv2.putText(img, name, (x1 + 6, y2 - 6),
   cv2.FONT_HERSHEY_COMPLEX, 1, (255, 255, 255), 2)
markAttendance(name)
cv2.imshow('Webcam', img)
key = cv2.waitKey(5)
if key == ord('q'):
break
cv2.destroyAllWindows
```